

CpE 484 Final Coding Project

Due: Thursday, April 30, 11:59PM

Dr. Zemerick

Spacecraft C&DH Mini Executive



FreeRTOS periodic task design, message queues, shared state, and rate-monotonic scheduling

In this final homework assignment, you will build a simplified spacecraft Command and Data Handling (C&DH) application in FreeRTOS. The software will model a small onboard computer that receives commands, updates spacecraft health data, and prints telemetry. You are not designing full flight computer software, instead you are implementing a compact real-time design with four periodic tasks, one command queue, and one shared spacecraft state structure.

1. Project Objectives and Learning Goals

- Build a small but realistic embedded application using FreeRTOS tasks, delays, and queues.
- Practice organizing software around periodic tasks and shared system state.
- Implement one simple message-passing path using a FreeRTOS command queue.
- Apply rate-monotonic and earliest-deadline-first scheduling ideas by assigning priorities based on task periods.
- Produce clear console output that shows commands, state changes, and telemetry over time.

2. System Overview

Your software represents a simplified spacecraft C&DH computer. It will run **four** periodic tasks. One task generates a fixed sequence of commands, one task receives and executes those commands, one task updates housekeeping data, and one task prints a telemetry line. Example C command definitions and the global spacecraft structure are provided for reference.

3. Task and Data-Flow Diagram

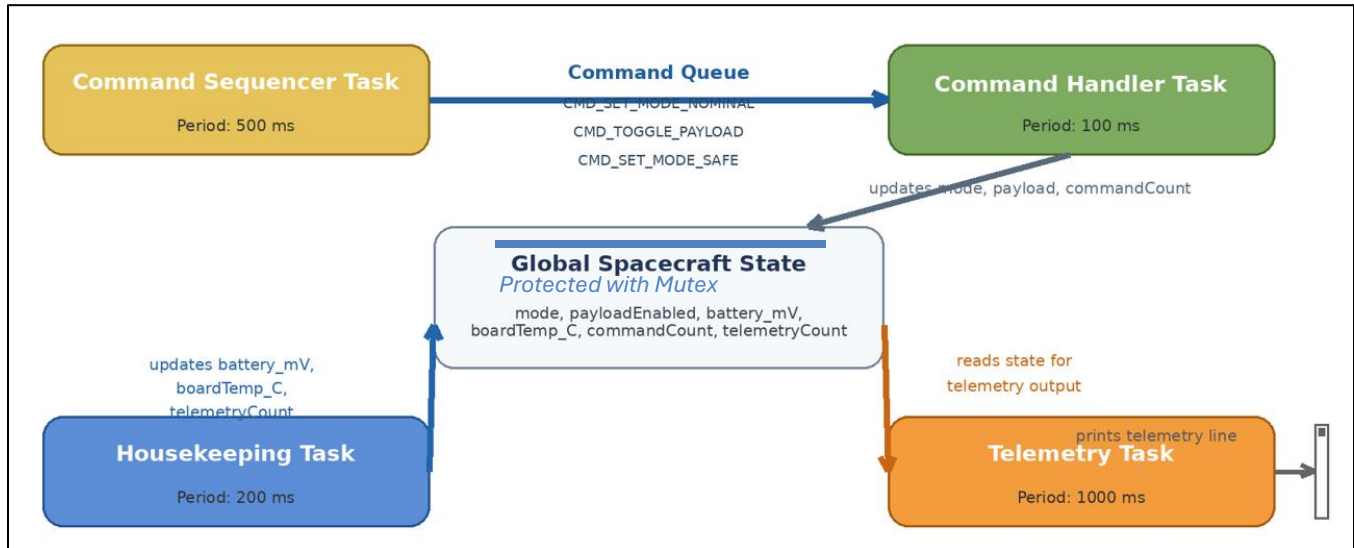


Figure 1. Required task, queue, and shared-state relationships.

4. Required Tasks

Task	Period*	Deadline	Required role	What it must do
CommandSequencerTask	500(0) ms	600(0) ms	Generate commands	Send the next command to the command queue and print what was queued. Cycle through each command iteratively. Every time the task wakes up and is running, have it send the next command.
CommandHandlerTask	100(0) ms	250(0) ms	Execute received commands	Read at most one queued command per activation/wakeup, update the global spacecraft state, and print the result. Use the message queue to block until a command is received. Use a switch/case statement to execute the commands received. This task will update the Spacecraft mode, command count, and payload enabled.

Task	Period*	Deadline	Required role	What it must do
HousekeepingTask	200(0) ms	210(0) ms	Update health data	Update battery and board temperature based on payload state and increment telemetryCount. For hardware/bus data, have this task generate random values for: battery (mV): between 1 and 250mV board temp (C) between -50 and 50 degC
TelemetryTask	100(0) ms	120(0) ms	Report telemetry	Read the global spacecraft state and print one formatted telemetry line.

* Since we are running in Linux/Window/Mac, you may need to lengthen the period and deadline in order to observe the system behavior. Simply multiply by a factor of 10. For example, if the period is 100ms, then make it 1000ms just to be able to effectively view the printf's.

5. Shared Data Structure Examples

Shared memory / global memory, that saves the state of the spacecraft.

Because multiple tasks access the global spacecraft state, you must protect gSpacecraft with a single FreeRTOS mutex named gStateMutex. Any task that reads or writes gSpacecraft must first take the mutex and must give it back immediately after the shared-data access is complete. The mutex must not be held during printf, queue operations, delays, or any other blocking call. FreeRTOS mutexes are the correct mechanism here because they are intended for mutual exclusion and support **priority inheritance**.

```
typedef struct
{
    SpacecraftMode_t mode;                // Updated by CommandHandlerTask
    uint8_t          payloadEnabled;      // Updated by CommandHandlerTask
    uint32_t          battery_mV;         // Updated by Housekeeping Task
    int32_t           boardTemp_C;        // Updated by Housekeeping Task
    uint32_t          commandCount;       // Updated by CommandHandlerTask - Counter
    uint32_t          telemetryCount;     // Updated by Housekeeping Task - Counter
} SpacecraftState_t;

SpacecraftState_t gSpacecraft;           // Global Data Store - Must be Protected with Mutex
```

6. Functional Requirements

6.1 CommandSequencerTask

- Run every 500 ms using a periodic task structure.
- Create a CommandMessage_t and send it to the command queue.
- Upon activation, send the next command.
- Print a short message showing which command was stored into the message queue.

Example code for the CommandMessage to put into the message queue.

For timeTag, you can increment the timeTag by one for each message.

```
typedef struct
{
    CommandType_t command;
    uint32_t       timeTag;
```

```
} CommandMessage_t;
```

Define exactly these three commands:

- CMD_SET_MODE_SAFE
- CMD_SET_MODE_NOMINAL
- CMD_TOGGLE_PAYLOAD

```
typedef enum
{
    CMD_SET_MODE_SAFE = 0,
    CMD_SET_MODE_NOMINAL = 1,
    CMD_TOGGLE_PAYLOAD = 2
} CommandType_t;
```

6.2 CommandHandlerTask

- Run every 100 ms.
- Check the command queue and execute at most one command per activation.
- For CMD_SET_MODE_SAFE: set mode = MODE_SAFE and force payloadEnabled = 0.
- For CMD_SET_MODE_NOMINAL: set mode = MODE_NOMINAL.
- For CMD_TOGGLE_PAYLOAD: if mode is MODE_NOMINAL, toggle payloadEnabled; if mode is MODE_SAFE, leave payload off and print "command rejected in SAFE mode".
- Increment commandCount only when a command is actually executed.

```
typedef enum
{
    MODE_SAFE = 0,
    MODE_NOMINAL = 1
} SpacecraftMode_t;
```

6.3 HousekeepingTask

- Run every 200 ms.
- If payloadEnabled is 1: battery_mV -= 5 and boardTemp_C += 1.
- If payloadEnabled is 0: battery_mV -= 1 and, if boardTemp_C > 20, decrement boardTemp_C by 1.
- Increment telemetryCount once per activation.

6.4 TelemetryTask

- Run every 1000 ms.
- Read the global spacecraft state and print one telemetry line.
- The telemetry line must include: current tick count, mode, payload state, battery voltage, board temperature, commandCount, and telemetryCount.

7. Queue Requirement

You must use exactly one FreeRTOS queue:

- Queue name: Command Queue
- Sender: CommandSequencerTask

- Receiver: CommandHandlerTask
- Queue item type: CommandMessage_t

8. Monotonic Scheduling Requirement

Use rate-monotonic scheduling. Assign each task a priority from 1 to 100, where 1 is the highest priority and 100 is the lowest priority. Include a short justification showing how task period influenced your priority choices.

9. Earliest Deadline First (EDF)

Use EDF scheduling. Assign each task a priority from 1 to 100, where 1 is the highest priority and 100 is the lowest priority. Include a short justification showing how task deadline influenced your priority choices.

10. Submission Requirements and Deliverables

- Your C source code and header files.
- A one-page task table listing each task, its period, and the priority you assigned for both RMS and EDF.
- A short paragraph explaining your rate-monotonic priority ordering.
- A short paragraph explaining your EDF priority ordering.
- A console log or screenshot showing the program running with RMS and EDF.

11. Constraints

- Use exactly 4 tasks.
- Use exactly 3 command types.
- Use exactly 1 FreeRTOS command queue.
- Do not add extra application tasks.
- Do not use interrupts for this assignment.
- Do not use software timers for this assignment.
- Use periodic task structure; **busy-waiting is not allowed.**
- Console output is sufficient; no hardware drivers are required.

12. Grading Rubric

Category	Weight	What is being evaluated
Correct task implementation	30%	All four tasks exist, run periodically, and perform the required behavior.
Message Queue and command handling	20%	The command queue is used correctly and the three commands behave exactly as specified.
Shared state and telemetry	20%	Global structure is used correctly and telemetry output reflects the current spacecraft state. Mutex used correctly.
Scheduling analysis	15%	Priorities are assigned and justified using rate-monotonic and EDF reasoning.
Code quality and readability	10%	Code is organized, readable, and uses clear names and comments.

Category	Weight	What is being evaluated
Submission completeness	5%	All requested files, screenshots/logs, and explanation are included.
	100 Points	

13. Submission Guidelines

- The program output clearly shows the command sequence, command execution, and changing telemetry.
- SAFE mode correctly disables the payload and rejects payload toggles.
- Telemetry fields update in a way that matches the required logic.
- The priority explanation is short, correct, and directly tied to task periods and deadlines.
- Code is clean and commented